

Assignment: Build a Dual-Path L1 Rollup (Optimistic & ZK)

Due Date: 11 April, 23:59

Submission Format: zip file

Overview

In this assignment, you will build a simplified Layer 1 (L1) smart contract that acts as the settlement layer for an off-chain Layer 2 (L2) Rollup.

Unlike traditional rollups that are either strictly Optimistic or strictly Zero-Knowledge (ZK), your contract must support a **dual-path challenge mechanism**. The off-chain Sequencer submits state roots, and challengers can dispute invalid roots using either:

- an interactive-style **Fraud Proof path** (Optimistic), or
- a **Validity Proof path** (ZK).

1. System Architecture and Lifecycle

You are provided with a starter repository containing interfaces, structs, and off-chain TypeScript definitions. Your implementation work is primarily in:

- `contracts/Rollup.sol`
- `lib/state.ts`
- `lib/merkle.ts`

1.1 Deposits and Sequencer Staking

- **Sequencer staking:** The Sequencer must maintain at least 1 ETH stake to submit batches.
- **Recovery after slash:** If slashed, Sequencer submissions are blocked until stake is restored via `topUpStake()`.
- **User deposits:** Users lock L1 funds through `deposit()`. This does *not* automatically mutate on-chain L2 state; off-chain crediting is done by the Sequencer.

1.2 Batch Submission

The Sequencer groups L2 transactions and submits resulting state roots to L1:

- **Single batches:** one L2 transaction.
- **Multi batches:** up to `MAX_BATCH_TX = 5` L2 transactions.
- **Canonical linkage:** each new batch must link to current canonical state via `parentRoot`.

1.3 Challenge Window and Dispute Resolution

After submission, each batch is challengeable only during `challengeWindow`.

Path A: Optimistic Fraud Proofs (`challengeBatch`, `challengeMultiBatch`) Challengers provide previous root, transaction data, and Merkle proofs. Your contract must:

1. verify ECDSA signature for each L2 transaction,
2. verify sender/receiver leaves against the relevant pre-state root,
3. recompute intermediate and/or final state roots correctly, and
4. slash Sequencer (pay 1 ETH to challenger) if submitted root is invalid.

Path B: ZK Validity Proofs (`challengeBatchZK`, `challengeMultiBatchZK`) Your contract must:

1. bind challenge context with

```
keccak256(abi.encode(batchId, prevRoot, submittedRoot, isMulti))
```

2. pass `proof` and `publicInput` to external `zkVerifier`,
3. slash Sequencer if verifier returns `true`.

1.4 Finalization and Withdrawals

If a batch remains unchallenged through the challenge window, it may be finalized. Users then withdraw by proving their leaf against the finalized state root.

2. Security Requirements and Edge Cases

Your implementation is graded on correctness *and* adversarial robustness.

1. **Signature malleability (EIP-2):** Reject signatures with high-s.
2. **Cumulative withdrawals:** Prevent cross-batch double-withdrawal by tracking cumulative withdrawn amount per account.
3. **Checks-Effects-Interactions:** Prevent reentrancy in ETH transfer paths.
4. **Merkle index correctness:** Respect index-dependent sibling ordering at every proof level.

3. Testing and Grading

You are provided a public test suite (`test/rollup.test.ts`) for baseline validation. Passing public tests is necessary but not sufficient.

Hidden Autograder Coverage

The hidden suite includes adversarial checks such as:

- randomized states and balances (anti-hardcoding),
- deep-index Merkle proofs,

- reentrancy attack attempts,
- high-**s** signature malleability attempts,
- boundary conditions (including zero-value edge handling),
- strict API routing and mode constraints.

4. Required Technical Report (Max 5 Pages)

Along with code, submit a brief technical report (**maximum 5 pages**, excluding references/appendix). Reports above the limit may receive a penalty.

Questions You Must Answer (all 5 required)

Your answers must be grounded in *your own implementation and experiments*. Generic theory-only answers will not receive full credit.

1. Optimistic vs ZK Challenge Path Trade-offs

Compare the two dispute mechanisms implemented in your system: security assumptions, on-chain cost profile, developer complexity, and failure modes. Include one concrete scenario where each path is preferable.

2. State Transition Correctness and Merkle Replay

Explain, with one worked example transaction, how your implementation validates sender/receiver leaves and recomputes intermediate/final roots. Clearly justify why **BalanceBefore** and **NonceBefore** are necessary.

3. Withdrawal Safety Across Multiple Finalized Batches

Describe how your contract prevents cross-batch double-withdrawals. Include a short attack narrative and explain why your chosen accounting strategy blocks it.

4. Signature and Reentrancy Hardening

Document your protections for ECDSA malleability and for reentrancy in withdrawal/challenge flows. Explain what would break if each protection were removed.

5. Evaluation of Your Design Under Adversarial Testing

Present at least two hidden-style edge cases you tested yourself (or simulated) beyond simple happy paths. For each, state expected behavior and actual observed outcome.

Report Evidence Requirements

Include concise evidence so graders can verify claims quickly:

- relevant function names and file paths (for example, `Rollup.sol`, `state.ts`),
- at least one test reference per question (public or self-authored),
- at most two small snippets/figures per question (focus on clarity, not volume).

Recommended Report Structure

1. System summary (0.5 page)
2. Q1–Q5 answers (about 3.5–4 pages total)
3. Key limitations and future improvements (0.5 page)

5. Implementation Guidance

- Implement deterministic off-chain logic first (`state.ts`, `merkle.ts`).
- Understand why `BalanceBefore`/`NonceBefore` are required for safe replay.
- Keep cryptographic and state logic explicit and readable.
- Avoid unnecessary external dependencies unless explicitly allowed by course staff.

Deliverables Checklist

- Completed `contracts/Rollup.sol`
- Completed `lib/state.ts`
- Completed `lib/merkle.ts`
- Passing public test suite
- Technical report answering all 5 required questions (**max 5 pages**)
- Submission package per course instructions

Good luck, and build securely.